CS121 Data Structures in C

05 *More on Linked Lists*

Outline

- Application

  Representation of single-variable polynomials
  - The types of the polynomials
  - Different representations for each type of polynomials
  - How to implement addition and multiplication

- Linked list extension

Type of Polynomials

- A *polynomial* consists of *terms:*

$$\boldsymbol{Polynomial(X)} = \sum_{i=0}^{N} \boldsymbol{A_i X^i}$$

$$\underbrace{3x^2}_{\text{term 1}} \underbrace{-5x}_{\text{term 2}} \underbrace{+4}_{\text{term 3}}$$

  - A term can be represented by its coefficient and exponent.

$$10\,x^{1000} + 5\,x^{14} + 1\,x^{0}$$

coefficient: 10; exponent: 1000

Type 1: most of the coefficients A_i are nonzero.

We can use a simple array to store the coefficients.

Type 2: most of the coefficients A_i are zero. Such as the above example.

We can use a singly linked list to represent the polynomial.

Polynomial Representation-type1

$$Polynomial(X) = \sum_{i=0}^{N} A_i X^i$$

- Most coefficients are nonzero

  Implementation of the polynomial based on array

```
//type declaration for array implementation of the polynomial
typedef struct
{
  int CoeffArray[ MaxDegree + 1 ]; //store the coefficients
  int HighPower;                   //the highest power(次幂)
} *Polynomial;
```

```
#define MaxDegree 100
```

Polynomial Representation-type1

$$Polynomial(X) = \sum_{i=0}^{N} A_i X^i$$

- Polynomial addition

```
//Initialization
ZeroPolynomial( Polynomial Poly )
{
  int i;

//MaxDegree:the maximum power
  for( i = 0; i <= MaxDegree; i++ )
       Poly->CoeffArray[ i ] = 0;

  Poly->HighPower = 0;
}
```

```
typedef struct
{
  int CoeffArray[ MaxDegree + 1 ];
  int HighPower;
} *Polynomial;
```

Polynomial Representation-type1

$$Polynomial(X) = \sum_{i=0}^{N} A_i X^i$$

$(4x^2 - 4) + (-x^2 - 4x + 4)$

- Polynomial addition

```
void AddPolynomial( const Polynomial Poly1, const Polynomial Poly2,
                    Polynomial PolySum )
{
   int i;
//Initialization
   ZeroPolynomial( PolySum );

//Decide the highest power of the result polynomial
   PolySum->HighPower = Max( Poly1->HighPower, Poly2->HighPower );

//add the corresponding terms of Poly1 and Poly2
   for( i = PolySum->HighPower; i >= 0; i-- )
      PolySum->CoeffArray[ i ] = Poly1->CoeffArray[ i ]
                                 + Poly2->CoeffArray[ i ];
}
```

Polynomial Representation-type1

$$Polynomial(X) = \sum_{i=0}^{N} A_i X^i$$

- Polynomial multipication

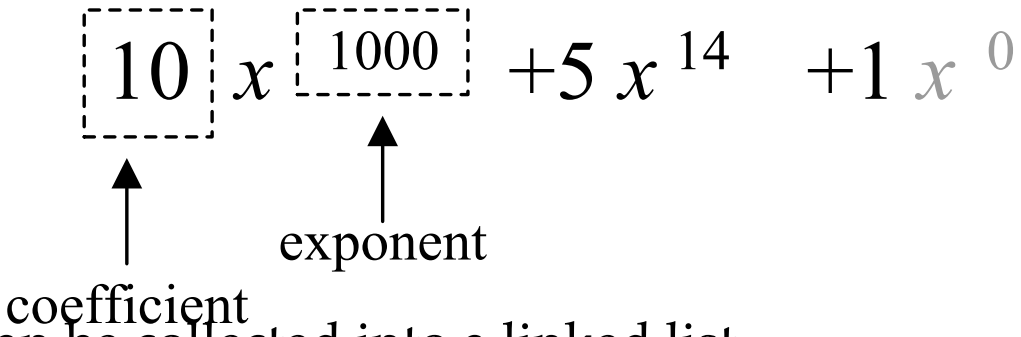
```
void MultPolynomial( const Polynomial Poly1, const Polynomial Poly2,
                     Polynomial PolyProd )
{
   int i, j;
 //Initialization
   ZeroPolynomial( PolyProd );
 //the highest power of the product is the addition of two
 //polynomials' powers
   PolyProd->HighPower = Poly1->HighPower + Poly2->HighPower;

   if( PolyProd->HighPower > MaxDegree ) //error checking
      Error( "Exceeded array size" );
   else                                  //calculate the coefficient
      for( i = 0; i <= Poly1->HighPower; i++ )
         for( j = 0; j <= Poly2->HighPower; j++ )
            PolyProd->CoeffArray[ i + j ] += Poly1->CoeffArray[ i ] *
                                             Poly2->CoeffArray[ j ];
}
```

Polynomial Representation

- Analysis
  - For type 1, the running time of the multiplication routine is proportional to the product of the degree of the two input polynomials. (ignoring the time of the initialization)
  - That is adequate for dense polynomials (most of the coefficients are nonzero)
  - For type 2, where most of the coefficients are zero, it wastes a lot of time on multiplying zero terms. That is unacceptable.
  - The solution is to use a singly linked list to represent the polynomials.

Polynomial Representation- type2

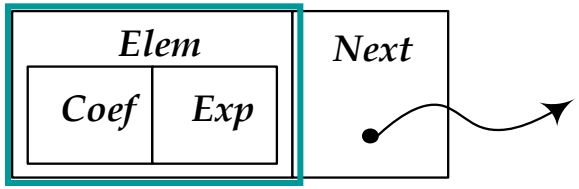
- A *polynomial* consists of *terms*.
  - A term can be represented by its coefficient and exponent.



  - Terms can be collected into a linked list.

- Each term representation: use compound elements

```
typedef  struct _T_term   term_t, elem_t;
struct _T_term {
     int Coef, Exp;
};
```

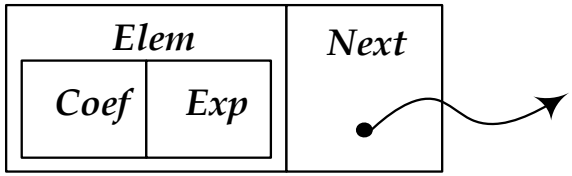


Polynomial Representation

- Polynomial representation: a linked list

  Each node:

```
typedef   struct  _T_node     node_t;

struct _T_node {
     elem_t Elem;
     node_t *Next;
};
```

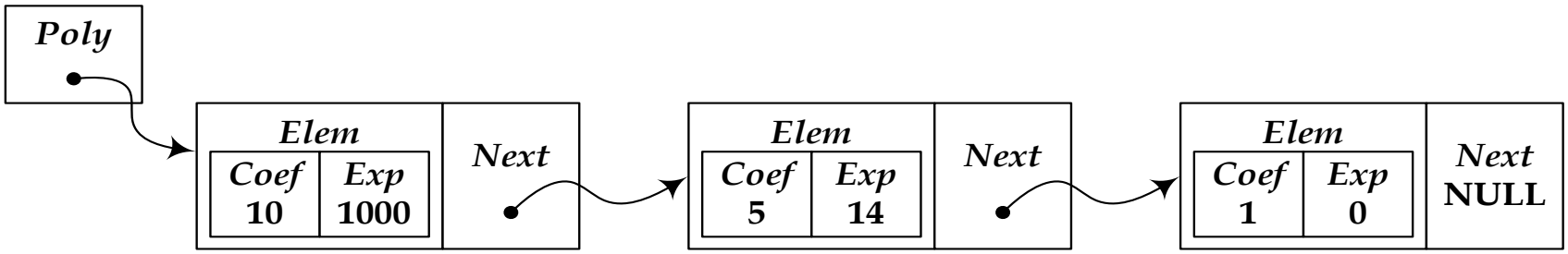


A polynomial:

$10\,x^{1000} + 5\,x^{14} + 1\,x^{0}$

coefficient, exponent

```
typedef  node_t  *poly;//a pointer points to type of node_t
```

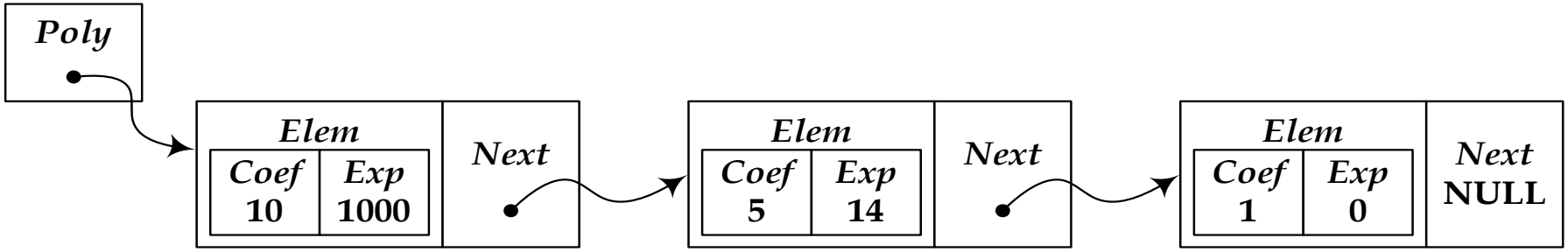


Polynomial Addition (1)

- We return a new linked list as the result of addition instead of updating the input.

```
poly Add_poly(poly A, poly B);
```

$$A = 10x^{1000} + 5x^{14} + 1$$
$$B = 4x^{5000} - 10x^{1000} + 12x^{14}$$
$$C = A + B = 4x^{5000} + 17x^{14} + 1$$



Polynomial Addition (2)

Steps

- Visit the terms of both polynomials sequentially.
  - **Case 1:** The term exponents of both polynomials are different.

    We copy the term with the **greater** exponent, and go to the next term of the corresponding polynomial.
  - **Case 2:** The term exponents of both polynomials are the same.

    We add the coefficients. There are two sub-cases.
    - case2.1: The addition of the two coefficients is zero. These two terms can be eliminated. We needn't do anything.
    - case2.2: The addition of the two coefficients is not zero. We add a new term with the additional result of the two coefficients and the corresponding exponent.
  - **Case 3:** If reaching one of the polynomials' end, we copy the rest of another to the end.

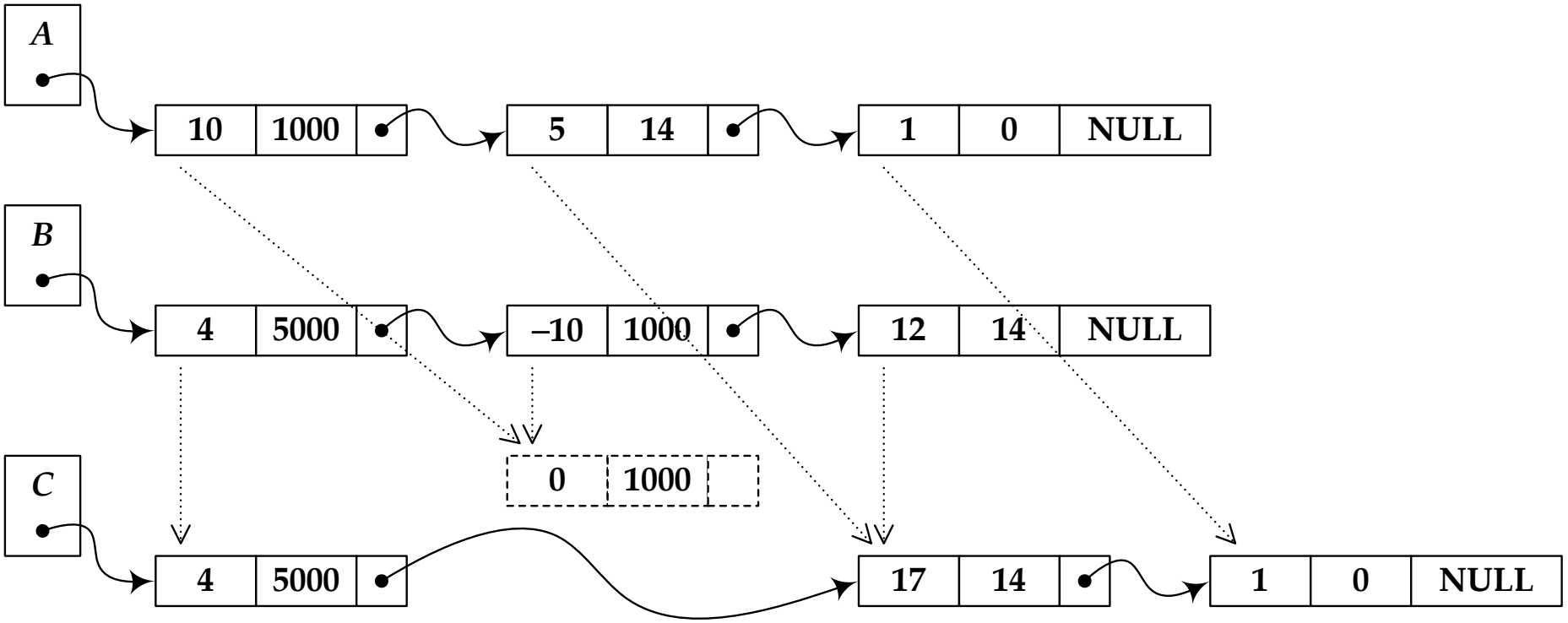
  And then, go to the next term of both polynomials.

$$A = 10x^{1000} + 5x^{14} + 1$$

$$B = 4x^{5000} - 10x^{1000} + 12x^{14}$$

$$C = A + B = 4x^{5000} + 17x^{14} + 1$$

Polynomial Addition (3-Fig.)



$$A = 10x^{1000} + 5x^{14} + 1$$

$$B = 4x^{5000} - 10x^{1000} + 12x^{14}$$

$$C = A + B = 4x^{5000} + 17x^{14} + 1$$

Polynomial Addition (4)

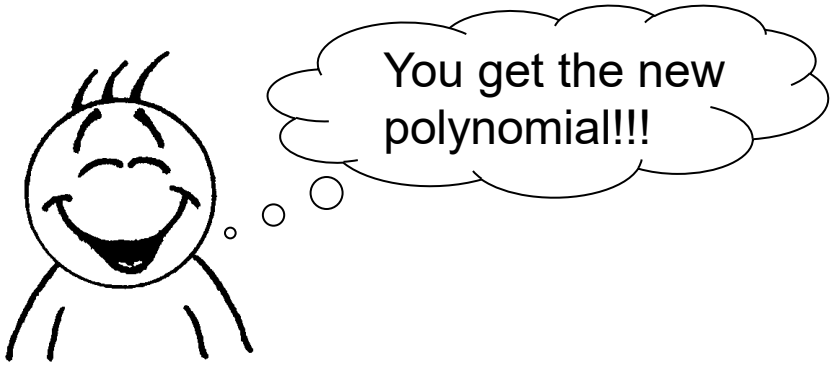
- A helper function:

  To create a new term , then return the address of its ***Next*** field, which can point to an other new term.

```
typedef  node_t  *poly;//a pointer points to type of node_t

poly *New_next(poly *PP, int Coef, int Exp)
{
    //allocate memory for the new term
    *PP = (poly)malloc(sizeof(**PP));
    if ( *PP == NULL )
    {
        printf("Out of space!!!");
    }
    //set the values of the new term
    (*PP)->Elem.Coef = Coef;
    (*PP)->Elem.Exp = Exp;

    //return "Next" of the new term
    return &((*PP)->Next);
}
```

```
poly Add_poly(poly A, poly B)
{
   poly C, *PC = &C;

   poly P = A, Q = B;
   while ( P && Q ) {  //not empty
      if ( P->Elem.Exp == Q->Elem.Exp ) {
        int S = P->Elem.Coef + Q->Elem.Coef; //add coefficients

        if ( S != 0 ) );
           PC = New_next(PC, S, P->Elem.Exp);

        P = P->Next, Q = Q->Next;
      } else {
          if ( P->Elem.Exp > Q->Elem.Exp ) {
             PC = New_next(PC, P->Elem.Coef, P->Elem.Exp);
             P = P->Next;
          } else {
             PC = New_next(PC, Q->Elem.Coef, Q->Elem.Exp);
             Q = Q->Next;
            }
        }
   }
/*  ......
```

```
…… Add_poly continues */

    for ( ; P; P = P->Next )
        PC = New_next(PC, P->Elem.Coef, P->Elem.Exp);

    for ( ; Q; Q = Q->Next )
        PC = New_next(PC, Q->Elem.Coef, Q->Elem.Exp);

    *PC = 0; /* NULL */

    return C;
}
```



Polynomial Construction

- Create a one-term poly.

```
poly Create_term(int Coef, int Exp)
{
    poly T = (poly)malloc(sizeof(*T));
    if ( T == NULL )
    {
        printf("Out of space!!!");
    }
    T->Elem.Coef = Coef;
    T->Elem.Exp = Exp;
    T->Next = 0;
    return T;
}
```

- Destructive addition. Destroy the input polynomials after the addition.

```
poly Add_poly_del(poly A, poly B)
{
    poly C = Add_poly(A, B);
    Destroy_poly(A);
    Destroy_poly(B);
    return C;
}
```

Polynomial Destruction and Printing

```
void Destroy_poly(poly A)
{
    poly P = A, Q;
    while ( P ) {
        Q = P->Next;
        free(P);
        P = Q;
    }
}


void Print_poly(poly A)
{
    poly P = A;
    for ( ; P; P = P->Next )
        printf("+%dx^%d ", P->Elem.Coef, P->Elem.Exp);
    printf("\n");
}
```

Create Polynomial

$$A = 10x^{1000} + 5x^{14} + 1$$

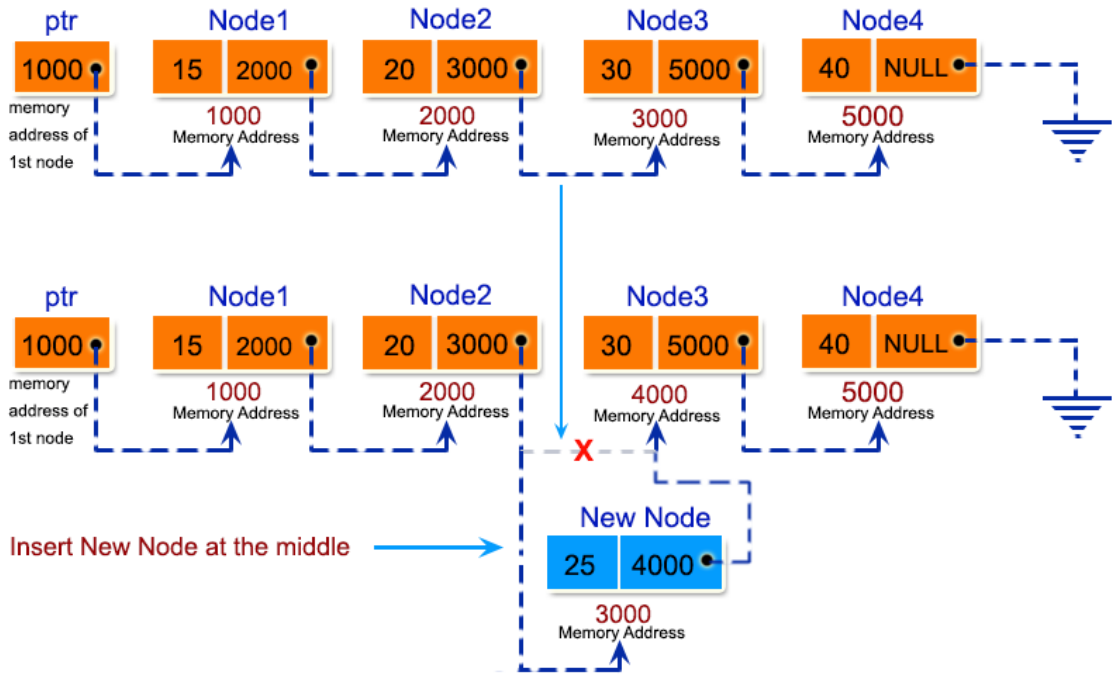
```
poly A = Add_poly_del(
            Add_poly_del(Create_term(10,1000),
                         Create_term(5,14)),
         Create_term(1,0) );
```

```
poly Add_poly_del(poly A, poly B);
poly Create_term(int Coef, int Exp);
```

- See the complete source code:
  "poly.c" (https://users.cs.fiu.edu/~weiss/dsaa_c2e/poly.c)

Assignment 04

- Using singly linked list ADT, to implement an algorithm to insert a new number in the middle of a singly linked list.



Assignment 04

- Submit it to Moodle
- Declaration of your function as:

```
void InsertMiddle( List L , int num);
```

a) To write a test program to print the list:

0,1,2,…,9

b) Then using your function to insert 10 and print the result.

You can assume that the list L has even number of elements.